DAY 4

Real-Time with Socket.IO

A second transport on the same service layer.

Recap: Where Day 3 Left You

Your project now has the layered shape:

```
task-api/
├── server-express.js      ← 30-line wiring file
├── logger.js              ← stream-based request log
├── package.json
├── routes/
│   └── taskRoutes.js      ← Express Router, one file per resource
├── services/
│   └── taskService.js     ← business rules, knows nothing about HTTP
├── storage/
│   └── fileStorage.js     ← the only file that touches the filesystem
├── data/
│   └── tasks.json
├── public/                ← empty. Today we fill it.
└── logs/
    └── server.log
```

It works, and it's clean. But it has one structural limitation that no amount of refactoring fixes:

> **Everything a client knows, it learned by asking. The server has no way to say anything on its own.**

Today we give the server a voice — and because of the Day 3 architecture, we do it without touching the service or storage layers. That's the architecture test running again, one day early.

1. The Problem: HTTP Can't Push

HTTP is strictly request → response:

```
Client: "GET /tasks"       →
                           ←  Server: "here they are"
(connection is done)
```

The server cannot start a conversation. If Browser A adds a task, Browser B has no idea until B asks again.

The workaround era: polling

```
// Browser B, refusing to be left out
setInterval(async () => {
  const res = await fetch("/tasks");
  renderTasks(await res.json());
}, 2000);
```

This 'works', but look at the trade:

Poll interval	Update latency (worst case)	Requests per client per hour
10 s	10 s — feels broken	360
2 s	2 s — feels sluggish	1,800
200 ms	instant-ish	18,000

And the punchline: almost every one of those responses says 'nothing changed.' You're paying full HTTP overhead — headers, routing, logging, JSON serialization — to learn nothing, thousands of times an hour, per client.

What we actually want:

```
Client and server hold ONE connection open.
Either side can send a message at any moment.
```

That's a WebSocket.

2. WebSockets: One Handshake, Then an Open Pipe

A WebSocket connection starts life as a normal HTTP request — which is why it can share port 3000 with your Express app. The client asks to upgrade, the server agrees, and from that point on the connection stops being HTTP entirely.

Here's how the WebSocket upgrade handshake works:

```
Client sends:
GET /socket.io/ HTTP/1.1
Host: example.com
Connection: Upgrade
Upgrade: websocket

Server responds:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade

<-- Persistent TCP connection established -->
<-- Bidirectional message flow -->
```

Key properties

- Bidirectional — server can push without being asked
- Persistent — one handshake, then the pipe stays open
- Low overhead — messages are small frames, not full HTTP requests
- ws:// and wss:// are the protocol schemes (like http:// and https://)

That 101 Switching Protocols response is the exact moment a connection leaves the HTTP world. You'll watch it happen in your own server.log in Lab 2.

3. Socket.IO: WebSockets with Batteries Included

You can use raw WebSockets in Node (the `ws` package). In practice, most Node apps use Socket.IO (4.x), which sits on top and solves the problems you'd otherwise solve yourself:

Concern	Raw WebSocket	Socket.IO
Connection drops (wifi, sleep)	You write reconnect logic	Automatic reconnection with backoff
Restrictive networks/proxies	Connection just fails	Falls back to HTTP long-polling
Message structure	Raw strings — you invent a format	Named events with JSON payloads
Send to everyone	Loop over clients yourself	`io.emit(...)` — one call
Groups of clients	Build it yourself	Rooms built in
"Did you get that?"	Build it yourself	Acknowledgement callbacks
Browser client library	Native WebSocket API	Served automatically at `/socket.io/socket.io.js`

Install (server side only — the browser client is served for you):

```
npm install socket.io
```

> ⚠ Caveat: Socket.IO speaks its own protocol on top of WebSockets. A Socket.IO server can't talk to a plain WebSocket client, and vice versa. Both ends must use Socket.IO. That's the price of the batteries.

Before we wire it in, we need one core Node concept the course hasn't formally met yet — and it turns out you've been using it since Day 2.

4. First, the Pattern: Node's EventEmitter

`events` is a built-in module. An `EventEmitter` lets one piece of code announce that something happened, and any number of other pieces react — without the announcer knowing who's listening.

```
const EventEmitter = require("events");

const emitter = new EventEmitter();

// Any number of independent listeners
emitter.on("order:placed", (order) => {
  console.log("Email service: confirm order", order.id);
});

emitter.on("order:placed", (order) => {
  console.log("Inventory service: reserve stock for", order.id);
});

// One announcement — both listeners fire, in registration order
emitter.emit("order:placed", { id: 42 });

// Output:
// Email service: confirm order 42
// Inventory service: reserve stock for 42
```

The API you need today:

```
emitter.on(name, fn);      // subscribe — fires every time
emitter.once(name, fn);    // subscribe — fires one time, then removes itself
emitter.off(name, fn);     // unsubscribe
emitter.emit(name, data);  // announce (listeners run synchronously, in order)
```

Two things worth knowing before they bite you:

- The "error" event is special. If something emits "error" and nobody is listening, Node throws and crashes the process. Streams do this — it's why unhandled stream errors kill servers.
- The MaxListeners warning. If one emitter accumulates more than 10 listeners for the same event, Node prints MaxListenersExceededWarning. That warning almost always means you're registering listeners in a loop or per-request instead of once — it's Node telling you you have a leak. You'll see exactly how this happens (and how to avoid it) in Section 7.

> You've been using EventEmitters since Day 2. Remember parseBody?
>
> ```
> req.on('data', chunk => (body += chunk));
> req.on('end', () => { ... });
> ```
>
> req IS an EventEmitter. So are streams, so is the server object — and so is every Socket.IO socket you're about to meet. Learn this one pattern and half of Node's API surface becomes familiar.

5. Wiring Socket.IO into the Express Server

The one gotcha everyone hits

Socket.IO needs to attach to the HTTP server object — the thing that owns the TCP port — not to the Express app (which is just a request handler function). Until now, app.listen() created that server invisibly. Today we make it explicit:

```
// BEFORE (Day 3) — Express creates a hidden http server
app.listen(PORT, () => console.log(`Server ready on http://localhost:${PORT}`));

// AFTER (Day 4) — we create the server ourselves and share it
const http = require("http");
const httpServer = http.createServer(app);   // Express handles HTTP requests...
const io = new Server(httpServer);           // ...Socket.IO handles the upgrade

httpServer.listen(PORT, () => console.log(`Server ready on http://localhost:${PORT}`));
```

⚠ The classic mistake: creating httpServer, attaching io to it — then calling app.listen(PORT) anyway. That starts a second, hidden server that Socket.IO isn't attached to. Every socket connection 404s, HTTP works fine, and you lose an hour. If sockets won't connect, check which object you called .listen() on.

server-express.js — Day 4 version

```
// server-express.js
const http = require("http");
const path = require("path");
const express = require("express");
const { Server } = require("socket.io");

const taskRoutes = require("./routes/taskRoutes");
const { logRequest } = require("./logger");

const app = express();
const httpServer = http.createServer(app);
const io = new Server(httpServer);

const PORT = process.env.PORT || 3000;

// --- Middleware (runs in order) ---------------------------------------
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));
app.use((req, _res, next) => { logRequest(req); next(); });

// --- Routes -----------------------------------------------------------
app.use("/tasks", taskRoutes);

app.get("/health", (req, res) => {
  res.json({ status: "ok", uptime: process.uptime() });
});

// --- 404 catch-all ----------------------------------------------------
app.use((req, res) => {
  res.status(404).json({ error: "Route not found" });
});

// --- Error handler (must be LAST; 4 args signal an error handler) --------
app.use((err, req, res, _next) => {
  console.error(`[${new Date().toISOString()}]`, err);
  res.status(500).json({ error: "Internal server error" });
});

// --- Socket.IO (temporary inline version — Section 7 extracts this) ------
io.on("connection", (socket) => {
  console.log("socket connected:", socket.id);

  socket.on("disconnect", (reason) => {
    console.log("socket disconnected:", socket.id, reason);
  });
});

httpServer.listen(PORT, () => {
  console.log(`Server ready on http://localhost:${PORT}`);
});
```

Notice what didn't change: every line of middleware, routing, and error handling from Day 3 is untouched. Socket.IO is bolted on beside Express, not into it.

The client side

Because io is attached to the server, the browser library is served automatically. Any page under public/ can do:

```
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io(); // connects to the server that served this page

  socket.on("connect", () => {
    console.log("connected as", socket.id);
  });
</script>
```

Watch the upgrade happen. Keep `tail -f logs/server.log` running while a page connects. You'll see one or two GET /socket.io/?EIO=4... lines — that's the handshake arriving as ordinary HTTP, passing through your logging middleware like any other request. Then: silence. The connection upgraded off HTTP, and messages now flow without ever touching Express, your middleware, or your logger again. Two transports, one port — and your HTTP middleware only sees one of them.

6. Talking: emit / on, and the Broadcast Cheat Sheet

Everything in Socket.IO is a named event with a payload — the EventEmitter pattern, stretched across the network:

```
// Server → one client
socket.emit("greeting", { text: "hello, you specifically" });

// Client
socket.on("greeting", (data) => console.log(data.text));
```

The part that matters for a collaborative app is choosing who receives:

Call	Who receives	Typical use
socket.emit("x", data)	That one client	Private reply, initial state
io.emit("x", data)	Every connected client	"The task list changed"
socket.broadcast.emit("x", data)	Everyone except that client	"Someone else is typing"
io.to("room").emit("x", data)	Clients who joined that room	Per-board channels (challenge)

Two practical warnings:

- Event names are just strings. Emit "task:create" and listen for "task:created" and nothing happens — no error, no warning, just silence. Typos here are the #1 source of "why isn't my listener firing."
- Debug tip: during development, log everything a socket receives:

```
socket.onAny((event, ...args) => console.log("→", event, args));
```

Naming convention for this course

We'll name events like resource:pastTenseVerb — task:created, task:updated, task:deleted, presence:update. Past tense on purpose: these events announce facts that already happened, not requests for something to happen. That distinction is the heart of the next section.

7. Second Transport, Same Service Layer

Here's today's real design question. When a task is added, someone has to call io.emit('task:created', task). Where does that line live?

Option A — emit inside the route handlers

```
// routes/taskRoutes.js — tempting, wrong
router.post("/", async (req, res) => {
  const task = await taskService.addTask(req.body.title);
  io.emit("task:created", task);   //
  res.status(201).json(task);
});
```

Two problems. The router now needs a reference to io (transport bleeding into transport). Worse: when Day 5's seed script — or any future caller — adds a task through the service without going through this route, no event fires. Clients silently miss updates.

Option B — import io inside the service

```
// services/taskService.js — worse
const io = require("../server-express"); //
```

This breaks the Day 3 contract outright — 'the service knows NOTHING about HTTP' — and it points a dependency arrow upward from the service to the entry point. Day 3 warned you exactly where that road ends: circular imports and a mysterious empty {}.

Option C — the service announces facts; the socket layer listens

The service doesn't need to know about Socket.IO. It only needs to say 'a task was created' — using the EventEmitter from Section 4. Whoever cares can subscribe.

services/taskEvents.js

```
// services/taskEvents.js
const EventEmitter = require("events");

// One shared emitter for task domain events.
module.exports = new EventEmitter();
```

Three lines. And it works because of something you learned on Day 3: modules are cached. Every file that requires taskEvents gets the same object — the service emits into it, the socket layer listens on it, and they never import each other. 'Module caching is why singletons just work in Node' — this file is that sentence, load-bearing.

services/taskService.js — the only change: three announcements

```
// services/taskService.js  (additions marked)
const storage = require("../storage/fileStorage");
const taskEvents = require("./taskEvents");                  // + NEW

async function addTask(title) {
  if (!title || !title.trim()) {
    throw new Error("Title is required");
  }
  const tasks = await storage.readTasks();
  const newTask = {
    id: Date.now(),
    title: title.trim(),
    done: false,
    createdAt: new Date().toISOString(),
  };
  tasks.push(newTask);
  await storage.writeTasks(tasks);
  taskEvents.emit("task:created", newTask);                  // + NEW
  return newTask;
}

async function updateTask(id, { title, done }) {
  const tasks = await storage.readTasks();
  const task = tasks.find(t => t.id === id);
  if (!task) throw new Error("Task not found");
  if (title !== undefined) task.title = title;
  if (done !== undefined) task.done = done;
  await storage.writeTasks(tasks);
  taskEvents.emit("task:updated", task);                     // + NEW
  return task;
}

async function deleteTask(id) {
  const tasks = await storage.readTasks();
  const before = tasks.length;
  const filtered = tasks.filter(t => t.id !== id);
  if (filtered.length === before) throw new Error("Task not found");
  await storage.writeTasks(filtered);
  taskEvents.emit("task:deleted", { id });                   // + NEW
}
```

Emit after the write succeeds, never before. If writeTasks throws, no event fires — clients only ever hear about things that actually happened. Announce facts, not intentions.

sockets/taskSockets.js — the bridge

```
// sockets/taskSockets.js
const taskEvents = require("../services/taskEvents");

function initTaskSockets(io) {
  // --- Domain events → broadcasts (registered ONCE, at startup) ---------
  taskEvents.on("task:created", (task) => io.emit("task:created", task));
  taskEvents.on("task:updated", (task) => io.emit("task:updated", task));
  taskEvents.on("task:deleted", (payload) => io.emit("task:deleted", payload));

  // --- Per-connection lifecycle -----------------------------------------
  io.on("connection", (socket) => {
    console.log("socket connected:", socket.id);
    io.emit("presence:update", { online: io.engine.clientsCount });

    socket.on("disconnect", () => {
      console.log("socket disconnected:", socket.id);
      io.emit("presence:update", { online: io.engine.clientsCount });
```

8. The Frontend: Commands In Over REST, Facts Out Over Events

Time to fill public/. One architectural decision first, because it shapes all the client code:

Mutations travel over REST. Rendering is driven only by socket events.

Why not do everything over the socket? Because your REST API already has everything a command needs — validation, status codes, the 400/404 error mapping, request logging, the centralized error handler. Rebuilding all of that in socket handlers would duplicate the whole Day 3 stack. So:

```
Load initial state:   GET /tasks          (once, on page load)
Change something:     POST / PUT / DELETE   (fire the command)
See any change:       socket events       (the ONLY thing that touches the UI)
```

This has a consequence that feels strange for about a minute: after a successful POST, the client does not render the new task. It waits ~10 ms for its own task:created broadcast — the same event every other tab gets. One source of truth, zero special cases, and the client renders the server's version of the task (with the server-generated id and createdAt), not its own guess. It's also why we broadcast with io.emit rather than socket.broadcast.emit: the originating tab needs the fact too.

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task Board</title>
  <link rel="stylesheet" href="/styles.css" />
</head>
<body>
  <main class="board">
    <header>
      <h1>Task Board</h1>
      <span id="presence" class="presence">— online</span>
    </header>

    <form id="add-form" autocomplete="off">
      <input id="title-input" type="text" placeholder="What needs doing?" required />
      <button type="submit">Add</button>
    </form>

    <ul id="task-list"></ul>
    <p id="empty-state" hidden>No tasks yet. Add the first one.</p>
  </main>

  <script src="/socket.io/socket.io.js"></script>
  <script src="/app.js"></script>
</body>
</html>
```

public/styles.css

```
:root {
  --bg: #111114;
  --card: #1b1b20;
  --text: #ececf0;
  --muted: #8a8a93;
  --accent: #7aa2ff;
  --danger: #ff6b6b;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, sans-serif;
  background: var(--bg);
  color: var(--text);
  display: flex;
  justify-content: center;
  padding: 3rem 1rem;
}

.board { width: 100%; max-width: 480px; }

header {
  display: flex;
  justify-content: space-between;
  align-items: baseline;
  margin-bottom: 1rem;
}

h1 { font-size: 1.4rem; font-weight: 600; margin: 0; }

.presence { color: var(--muted); font-size: 0.85rem; }

#add-form { display: flex; gap: 0.5rem; margin-bottom: 1.25rem; }

#add-form input {
  flex: 1;
  padding: 0.6rem 0.8rem;
  border: 1px solid #2c2c33;
  border-radius: 8px;
  background: var(--card);
  color: var(--text);
  font-size: 0.95rem;
}

#add-form button {
  padding: 0.6rem 1rem;
  border: none;
  border-radius: 8px;
  background: var(--accent);
  color: #0c1020;
  font-weight: 600;
  cursor: pointer;
}

#task-list { list-style: none; margin: 0; padding: 0; }

.task {
```

9. Presence and the Connection Lifecycle

You already wired presence in taskSockets.js; here's what's happening:

- `io.on("connection", socket => ...)` fires once per client that connects. Each socket is that client's private, long-lived channel (and yes — it's an EventEmitter).
- `socket.on("disconnect", ...)` fires when that client leaves — tab closed, wifi dropped, laptop lid shut.
- `io.engine.clientsCount` is the live number of connected clients — no manual counting needed.
- On both connect and disconnect we `io.emit("presence:update", ...)` so every tab's counter stays current.

> It counts connections, not people. Two tabs in one browser = 2. Refresh a tab and you'll see the number dip and recover — disconnect fires, then a fresh connection arrives. Watching that flicker teaches you the lifecycle better than any diagram.

10. Common Mistakes & Debugging

Problem	First thing to check
Sockets 404 / never connect	Did you call `app.listen` instead of `httpServer.listen`? Socket.IO is attached to `httpServer`.
`io` is not defined in the browser	Missing `<script src="/socket.io/socket.io.js">` before `app.js`.
Listener never fires, no error	Event name typo — names are strings. Add `socket.onAny(console.log)` and look at what's actually arriving.
Every event arrives N times	Domain-event subscriptions registered inside `io.on("connection")`. Move them to startup. See the trap in Section 7.
MaxListenersExceededWarning	Same bug as above — you're adding listeners per connection/request.
New task appears twice in the origin tab	Rendering both locally after POST and on the broadcast. One source of truth.
Task added via curl doesn't broadcast	Emits placed in the route handler instead of the service — only browser-originated changes fire. Emit from the service.
Works on `localhost:3000`, fails opening `index.html` from a `file://` path	Serve the page through `express.static` so page and socket share an origin.
Changes stop broadcasting after an error	Check the server console — a thrown error inside a `taskEvents.on` listener propagates from the synchronous emit inside your service call. Keep listeners tiny.

Lab 1: The Task Board (REST Only)

Goal

Build the browser UI for your task API using only what you already have — express.static and the REST endpoints. No sockets yet. We build the pain first, then fix it.

Concepts Practiced

- express.static serving a frontend
- fetch for GET / POST / PUT / DELETE
- DOM rendering from an array of tasks

Starter Setup

Create in public/:

- index.html (from Section 8 — but leave out both script tags for socket.io and replace them with only `<script src="/app.js"></script>`)
- styles.css (from Section 8)
- app.js — Section 8's version, minus the `const socket = io()` line and the entire `socket.on(...)` block

Requirements

1. On page load, GET /tasks and render the list
2. The form adds a task via POST /tasks
3. The checkbox toggles done via PUT /tasks/:id
4. The 🗑 button deletes via DELETE /tasks/:id
5. Since there are no events yet, refresh state manually — after each successful mutation, call `loadTasks()`:

```
// Lab 1 versions of the mutation handlers
formEl.addEventListener("submit", async (e) => {
  e.preventDefault();
  const title = inputEl.value.trim();
  if (!title) return;
  await fetch("/tasks", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title }),
  });
  inputEl.value = "";
  await loadTasks();   // ← temporary. Lab 4 deletes this line.
});
```

Test Cases

```
npm start
# Browser: http://localhost:3000
```

1. Add three tasks, toggle one, delete one — everything works in the tab you're using
2. Now open a second tab. Add a task in tab A. Look at tab B.

Acceptance Criteria

- Full CRUD works from the browser
- logs/server.log shows your clicks arriving as HTTP requests
- Tab B does not show tab A's changes until you refresh it — this is the expected result, and it's the whole point

Hints

- express.static was already wired on Day 3 — if the page 404s, check the middleware line exists and the files are really in public/
- Send ids in URLs as-is; URLs are strings anyway

Reflection

Tab B is connected to the same server as tab A. Why can't the server just tell tab B what happened? What exactly, in the HTTP model, is missing?

Lab 2: Attach Socket.IO

Goal

Get a live socket connection between browser and server, and watch the protocol upgrade happen in your own logs.

Concepts Practiced

- http.createServer(app)
- attaching a Socket.IO Server
- the connect / disconnect lifecycle

Requirements

1. npm install socket.io
2. Refactor server-express.js to the Section 5 shape: explicit httpServer, io attached, httpServer.listen at the bottom
3. Add the inline connection logger from Section 5 (connection + disconnect console logs)
4. In index.html, add <script src="/socket.io/socket.io.js"></script> above app.js
5. At the top of app.js: const socket = io(); plus a socket.on("connect", ...) log

Test Cases

```
npm start
# Terminal 2:
tail -f logs/server.log
```

Open and close tabs. Refresh. Close the laptop lid if you're feeling scientific.

Acceptance Criteria

- Server console logs socket connected: <id> per tab, and disconnected when a tab closes
- Browser console logs its own socket id
- server.log shows a couple of GET /socket.io/... lines at connect time — then nothing socket-related ever again, even while connections stay alive

Hints

- Sockets 404? You called .listen on the wrong object. Section 5, first code block.
- Each refresh produces a new socket id — connections don't survive page loads (reconnection logic gives you a new socket, not the old one back)

Reflection

Those /socket.io/ lines passed through your logging middleware like any request — then stopped, while the connection kept working. Where did the traffic go, and why can't Express middleware see it anymore?

Lab 3: Domain Events → Broadcasts

Goal

Make the service announce task:created / task:updated / task:deleted, and build the socket bridge that turns those announcements into broadcasts. No UI changes yet — we'll prove it works with curl and a browser console.

Concepts Practiced

- EventEmitter as a shared, cached module
- decoupling the service from every transport
- registering listeners once vs per-connection

Requirements

01

Create services/taskEvents.js (Section 7 — three lines)

02

Add the three taskEvents.emit(...) calls to services/taskService.js — each one after its writeTasks succeeds

03

Create sockets/taskSockets.js (Section 7). Move the connection/disconnect logging here from server-express.js

04

In server-express.js, replace the inline io.on("connection", ...) block with initTaskSockets(io)

Test Cases

```
// Browser tab, DevTools console:
socket.onAny((event, ...args) => console.log("→", event, args));
```

```
# Terminal — mutate the data WITHOUT the browser:
curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"added from curl"}'
```

Acceptance Criteria

- The curl command makes → task:created {...} appear in the browser console — a change made by one client, pushed to another, through the service layer
- task:updated and task:deleted arrive for PUT/DELETE curls too
- With 3 tabs open, each event arrives in each tab exactly once

Hints

- Events arriving N times = subscriptions inside the connection handler. Re-read the trap in Section 7.
- The service imports ./taskEvents, the sockets module imports ../services/taskEvents — same cached object. Nobody imports sockets/.

Reflection

The route handlers didn't change at all in this lab, yet every mutation now reaches every browser. Which Day 3 rule made that possible — and what would this lab have looked like in the Day 2 monolith?

Lab 4: The Live Board

Wire the frontend to render from socket events. This is the lab the Day 3 deck promised: add a task in one tab, watch it appear in the other before you've released the mouse.

Concepts Practiced

- event-driven rendering
- single source of truth
- idempotent state updates by id

Requirements

01	02	03
In app.js, add the three socket.on("task:*", ...) handlers from Section 8	Delete every await loadTasks() you added after mutations in Lab 1. loadTasks() now runs exactly once, at page load	Keep id comparisons as String(...) === String(...)

Test Cases

1. Two tabs side by side. Add in A → appears in B. Toggle in B → strikes through in A. Delete in A → gone in B.
2. The curl commands from Lab 3 — tasks appear in both tabs
3. Add a task and watch the origin tab closely: the task appears once, not twice

Acceptance Criteria

- All mutations propagate to every open tab without any refresh
- The originating tab shows each change exactly once
- Changes made via curl render in the browser too

Hints

- Task appears twice in the origin tab? You left a local render in a mutation handler. One source of truth — the broadcast.
- Nothing renders at all? socket.onAny(console.log) — are events arriving? If yes, your handler names have a typo. If no, back to Lab 3.

Reflection

After clicking Add, your own tab technically waits for the server's broadcast before showing the task. What did we gain by accepting that tiny delay? What could go wrong in a design where the client renders its own guess immediately?

Lab 5: Presence

Goal

Show a live 'N online' counter in every tab, driven by the connection lifecycle.

Concepts Practiced

- connection / disconnect events
- io.engine.clientsCount
- broadcasting state changes about the clients themselves

Requirements

01

In sockets/taskSockets.js, emit presence:update with { online: io.engine.clientsCount } on every connect AND every disconnect (Section 7 has the exact code)

02

In app.js, handle presence:update and write the count into the #presence element

Test Cases

1. Open 3 tabs — every tab says 3 online
2. Close one — the remaining two update to 2 online without any interaction
3. Refresh a tab and watch the counter carefully

Acceptance Criteria

- The counter is identical in all tabs at all times
- Opening/closing tabs updates every tab within a moment

Hints

- Emitting only inside connection means closing a tab never updates anyone — the disconnect side matters too
- io.emit (everyone) — not socket.emit (just the newcomer)

Reflection

On refresh the counter briefly dips before recovering. What two lifecycle events did you just watch, in what order? And what does clientsCount actually count — people, browsers, or something else?

Optional Challenge: Reconnection UX

Socket.IO reconnects automatically — but your app's state doesn't. Any event broadcast while a client was disconnected is simply gone. Handle it.

Requirements

1. Add a small banner element to index.html (e.g. `<div id="conn-banner" hidden>Reconnecting…</div>`)
2. Show it on `socket.on("disconnect", …)`, hide it on `socket.on("connect", …)`
3. The important part: on every reconnect, call `loadTasks()` to resync — because broadcasts you missed while offline are not replayed

```
socket.on("disconnect", () => { bannerEl.hidden = false; });

socket.on("connect", () => {
  bannerEl.hidden = true;
  loadTasks();   // resync: events during the gap are lost forever
});
```

Test

Stop the server (Ctrl+C) with two tabs open → banners appear. Restart it → banners clear on their own (that's the auto-reconnect). Now test the gap: stop the server, curl won't work while it's down, so instead — with tab B's network throttled to 'Offline' in DevTools — add tasks from tab A, bring B back online, and confirm B catches up.

Bonus ideas

- Typing indicator — on input, `socket.emit("typing")`; server relays with `socket.broadcast.emit` (everyone except the typist — finally a use for it); hide after a 1.5 s timeout
- Rooms — support `/board/:name` pages; `socket.join(name)` on connect and swap `io.emit` for `io.to(name).emit` in the bridge
- Commands over the socket — reimplement Add as `socket.emit("task:add", { title }, ack)` with an acknowledgement callback. Then write three sentences on what you lost versus REST (hint: status codes, express.json, the logger, the error handler) and when the trade is worth it anyway

Key Takeaways

HTTP answers; WebSockets announce

One upgrade handshake turns a request/response protocol into an open, bidirectional pipe on the same port.

EventEmitter is the pattern under everything

req, streams, sockets. on / emit, listeners run in order, and the MaxListeners warning means you're leaking subscriptions.

The service announces facts; transports subscribe

Three emit calls were the only change to business logic. Routes untouched, storage untouched, no layer imports upward, no circular requires.

Commands in over REST, facts out over events

The client's only source of UI truth is the broadcast — which is exactly why every tab, including the one that clicked, always agrees.

Register domain listeners once at startup

Per-socket handlers go inside connection. Mixing these up is the bug you will meet again in the wild.

Bridge to Day 5: Your app now has two transports, live collaboration, and a clean layered core. It also has a JSON file pretending to be a database. Here's the crack: every save rewrites the entire file. Day 3's atomic .tmp-then-rename trick means the file is never corrupted — but it does nothing about two requests that both read, both modify, and both write. Last writer wins; the other task silently vanishes. Add ids from Date.now() colliding in the same millisecond, no way to query without loading everything, and exactly one machine that can ever hold the file — and you have a storage layer that dies the moment two users get enthusiastic. Tomorrow we prove the data loss with a ten-line stress script, then make it impossible: MongoDB — behind the same storage interface. The Day 3 deck promised the swap would be one require line. Tomorrow we collect.